

LAB GUIDE WITH SOLUTIONS

Lab Guide

Functions

Ten labs with full solutions · approx. 6 hours

Elixir Training Programme

Elixir · functions

Session 7 · Instructor-led · Elixir 1.20.4 on Erlang/OTP 29

How to use this guide

Ten labs, in order, roughly six hours including discussion. Each lab states a goal, gives you the tasks, and then gives the full solution with the real output it produced on the training machine.

Session 6 declared `def` and `defp` and explained neither. This session is what they mean: named and anonymous functions, clauses and guards, captures and closures, the pipe, `map`, `filter` and `reduce`, recursion, dynamic calls — and the two abstractions built out of functions, behaviours and protocols. Macros are session 8.

#	Lab	Minutes	Covers
1	Named functions	30	<code>def</code> , <code>defp</code> , name/arity, default arguments
2	Clauses and guards	35	patterns in the head, clause order, <code>when</code>
3	Anonymous functions	30	<code>fn</code> , the dot, multi-clause <code>fn</code> , functions as data
4	The capture operator	30	<code>&Mod.fun/arity</code> , <code>&(&1)</code> , the three rules
5	Closures and higher-order functions	35	what a closure holds, factories
6	<code>map</code> , <code>filter</code> , <code>reduce</code>	35	Enum's big three, and <code>reduce</code> underneath
7	The pipe and composition	35	<code> ></code> , <code>then</code> , <code>tap</code> , pipelines of functions
8	Recursion and tail recursion	40	accumulators, the stack, measured
9	Dynamic calls and behaviours	40	<code>apply/3</code> , safe dispatch, <code>@callback</code> , <code>GenServer</code>
10	Capstone: a pricing engine	50	a protocol, closures, config, a behaviour

IMPORTANT Windows users: the launchers are `iex.bat` and `mix.bat`. In PowerShell, plain `iex` collides with PowerShell's own `Invoke-Expression` alias — always type `iex.bat`.

WATCH OUT Three starter files sit in `code/`. `function_lab.exs` carries labs 1 to 4; `recursion.exs` is lab 8 and gives you a stack-measuring helper; `pricing.exs` is the capstone. All three run as handed out — they print `:todo` where your code goes.

ERRORS ON PURPOSE Several tasks ask you to compile something that FAILS. Those are not mistakes in the guide — the error message is the lesson, and it is printed in full, in red. Read it before you read the explanation underneath.

LAB 1 · 30 MINUTES · NAMED FUNCTIONS

Named functions

GOAL Learn a function's real name. It is not `area` — it is `area/1`, and the arity is as much a part of the name as the letters are.

Task 1.1 — Public, private, and who can see what

In `function_lab.exs`, write `Temp.c_to_f/1` and `Temp.f_to_c/1`. Both round through ONE private helper `round1/1`. Then ask the module what it exports, and try calling the helper from outside.

Solution

```
defmodule Temp do
  def c_to_f(c), do: round1(c * 9 / 5 + 32)
  def f_to_c(f), do: round1((f - 32) * 5 / 9)
  defp round1(x), do: Float.round(x / 1, 1)
end

Temp.c_to_f(100)    = 212.0
Temp.c_to_f(-40)    = -40.0
Temp.f_to_c(98.6)   = 37.0
Temp.__info__(:functions) = [c_to_f: 1, f_to_c: 1]
function_exported?(Temp, :round1, 1) = false

apply(Temp, :round1, [1.25])
** (UndefinedFunctionError) function Temp.round1/1 is undefined or private
```

WHY `round1/1` is not hidden — it is absent from the module's export table, so there is nothing for another module to call. The error cannot tell "private" from "does not exist" because, from outside, there is no difference. The `x / 1` in `round1` is belt and braces: `Float.round/2` only takes floats — `Float.round(3, 1)` is a `FunctionClauseError` — and `/` always returns one.

Task 1.2 — When does the compiler notice?

Add a second module that calls `Temp.round1/1` directly, and a function that reaches it through `apply/3`. Predict which one the compiler complains about.

```
defmodule Caller do
  def direct, do: Temp.round1(1.0)
  def dynamic, do: apply(Temp, :round1, [1.0])
end

compiled - both modules loaded
warning: Temp.round1/1 is undefined or private
7 | def direct, do: Temp.round1(1.0)
  -> Caller.direct/0
```

WHY The direct call is a WARNING, not an error: the file compiles, both modules load, and the warning is printed once the compiler can see every module in the file. The `apply/3` call gets nothing at all — the function name is data, so the compiler cannot check it. Both fail the same way at run time. Treat that warning as an error.

Task 1.3 — Same name, different arity

Solution

```
defmodule Area do
  def area(side), do: side * side
  def area(w, h), do: w * h
end

Area.area(4)      = 16
Area.area(4, 5)   = 20
Area.__info__(:functions) = [area: 1, area: 2]
(&Area.area/1) == (&Area.area/2) = false
```

WHY Two entries in the export table and two different captured functions. They share a word and nothing else — you could delete one without touching the other. This is why the docs, the errors and the capture operator all say name/arity.

Task 1.4 — A default argument

Solution

```
defmodule Hello do
  def greet(name, greeting \\ "Hello"), do: "#{greeting}, #{name}!"
end

Hello.greet("Ada")      = "Hello, Ada!"
Hello.greet("Ada", "Hi") = "Hi, Ada!"
Hello.__info__(:functions) = [greet: 1, greet: 2]
```

WHY One definition, two public functions. `greet/1` is generated for you and simply calls `greet/2` with the default filled in — which is why it appears in the export table.

Task 1.5 — Defaults on a function with two clauses

Write `Weight.fmt/2` with a default unit, as two clauses guarded by `is_integer` and `is_float`, and put the default on both clauses.

```
defmodule Weight do
  def fmt(n, unit \\ "kg") when is_integer(n), do: "#{n} #{unit}"
  def fmt(n, unit \\ "kg") when is_float(n), do: "#{Float.round(n, 1)} #{unit}"
end

error: def fmt/2 defines defaults multiple times. Elixir allows defaults to be
declared once per definition. Instead of:

  def foo(:first_clause, b \\ :default) do ... end
  def foo(:second_clause, b \\ :default) do ... end

one should write:

  def foo(a, b \\ :default)
  def foo(:first_clause, b) do ... end
  def foo(:second_clause, b) do ... end

the previous clause is defined on line 2
```

The error tells you the fix. Do it:

Solution

```
defmodule Weight do
  def fmt(n, unit \\ "kg")
  def fmt(n, unit) when is_integer(n), do: "#{n} #{unit}"
  def fmt(n, unit) when is_float(n), do: "#{Float.round(n, 1)} #{unit}"
end

fmt(3) = "3 kg"
fmt(2.25, "lb") = "2.3 lb"
```

WHY A bodiless clause is a function HEAD: it declares the parameters and their defaults and nothing else. Defaults belong to the function, not to a clause, so they are declared exactly once.

CHECKPOINT You can say how many functions `def f(a, b \\ 1, c \\ 2)` defines, and why `f/1` and `f/2` have nothing to do with each other.

LAB 2 · 35 MINUTES · CLAUSES

Clauses and guards

GOAL Let the function head do the deciding. A clause is a pattern plus an optional guard, tried top to bottom — and when none matches, that is information, not a bug to hide.

Task 2.1 — One clause per shape

Solution

```
defmodule Shape do
  def area({:square, s}), do: s * s
  def area({:rect, w, h}), do: w * h
  def area({:circle, r}), do: Float.round(:math.pi() * r * r, 2)
end

Shape.area({:square, 3}) = 9
Shape.area({:rect, 2, 5}) = 10
Shape.area({:circle, 1}) = 3.14
```

Now call it with a shape nobody planned for:

```
Shape.area({:triangle, 3, 4})

** (FunctionClauseError) no function clause matching in Shape.area/1

    The following arguments were given to Shape.area/1:

        # 1
        {:triangle, 3, 4}
```

WHY The error prints the arguments. That is the first thing to read: it tells you exactly what arrived that you did not plan for. Resist adding `def area(_, do: 0` — a triangle quietly reporting an area of zero is worse than a crash.

Task 2.2 — Guards

Write `Grade.letter/1`: "A" for 90 and up, "B" for 75 and up, "C" for 50 and up, otherwise "F" — for integers 0 to 100 only. Anything else is an error tuple, decided in the head.

Solution

```
defmodule Grade do
  def letter(n) when not is_integer(n), do: {:error, :not_an_integer}
  def letter(n) when n < 0 or n > 100, do: {:error, :out_of_range}
  def letter(n) when n >= 90, do: {:ok, "A"}
  def letter(n) when n >= 75, do: {:ok, "B"}
  def letter(n) when n >= 50, do: {:ok, "C"}
  def letter(_, do: {:ok, "F"})
end

Grade.letter(95) = {:ok, "A"}
Grade.letter(90) = {:ok, "A"}
Grade.letter(80) = {:ok, "B"}
Grade.letter(50) = {:ok, "C"}
Grade.letter(12) = {:ok, "F"}
```

```
Grade.letter(101) = {:error, :out_of_range}
Grade.letter(-1) = {:error, :out_of_range}
Grade.letter(88.5) = {:error, :not_an_integer}
Grade.letter("90") = {:error, :not_an_integer}
```

WHY The order is the logic. The type check comes first, so every later clause may assume an integer; the range check second, so the letter clauses may assume 0 to 100; and each letter clause may assume every clause above it failed. The last one needs no guard at all.

Task 2.3 — Put the catch-all first

```
defmodule Grade do
  def letter(_), do: {:ok, "F"}
  def letter(n) when n >= 90, do: {:ok, "A"}
end

warning: this clause for letter/1 cannot match because a previous clause at
line 2 always matches

warning: the following clause is redundant:
  def letter(n) when n >= 90
it has type:
  dynamic()
previous clauses have already matched on the following types:
  term()

Grade.letter(95) = {:ok, "F"}
```

WHY Two warnings from two different parts of the compiler. The first is the classic pattern check. The second comes from the type checker Elixir 1.20 ships: it reasons that the first clause accepts `term()`, which already contains every value the second could see. And notice it still COMPILES — a 95 gets an F.

Task 2.4 — An exception inside a guard

Solution

```
defmodule Long do
  def long?(x) when length(x) > 2, do: true
  def long?(_), do: false
end

Long.long?([1, 2, 3]) = true
Long.long?([1])      = false
Long.long?("abc")    = false

length("abc") # outside a guard
** (ArgumentError) errors were found at the given arguments:
```

WHY `length("abc")` raises when you call it in a body. Inside a guard, the same exception just means "this clause does not match", and Elixir moves on to the next one. That is useful — the guard doubles as a type check — and dangerous, because a typo in a guard fails silently instead of loudly.

Task 2.5 — A function that is not allowed in a guard

```
defmodule Name do
```

```
def short?(s) when String.length(s) < 4, do: true
def short?(_), do: false
end

error: cannot invoke remote function String.length/1 inside a guard
```

Rewrite it with a guard-safe function:

Solution

```
defmodule Name do
  def short?(s) when is_binary(s) and byte_size(s) < 4, do: true
  def short?(_), do: false
end

Name.short?("ada")    = true
Name.short?("grace") = false
Name.short?("aÑá")   = false    byte_size is 5, not 3
```

WHY Guards may only call a closed set of functions, so they are guaranteed fast and free of side effects. `String.length/1` walks the string counting graphemes, so it is not in the set. `byte_size/1` is — but it counts BYTES, and "aÑá" is three characters in five bytes. If you need characters, move the test into the body.

Task 2.6 — Matching maps in the head

Solution

```
defmodule Resp do
  def handle(%{status: 200, body: body}), do: {:ok, body}
  def handle(%{status: s}) when s in 400..499, do: {:client_error, s}
  def handle(%{status: s}) when s >= 500, do: {:server_error, s}
  def handle(other), do: {:unexpected, other}
end

Resp.handle(%{status: 200, body: "hi"}) = {:ok, "hi"}
Resp.handle(%{status: 404}) = {:client_error, 404}
Resp.handle(%{status: 503}) = {:server_error, 503}
Resp.handle(%{status: 302}) = {:unexpected, %{status: 302}}
Resp.handle(:nope) = {:unexpected, :nope}
```

WHY A map pattern matches if the listed keys are present with matching values — other keys are ignored. Here a catch-all IS right: this function is the boundary where an HTTP response becomes a tagged tuple, and it can say what an unexpected one is.

CHECKPOINT You can explain why `Grade.letter/1` gave an F to a 95, and what `hd([])` means when it appears inside a guard.

LAB 3 · 30 MINUTES · FUNCTIONS AS VALUES

Anonymous functions

GOAL Treat a function as a value: bind it, call it, put it in a map, return it from another function.

Task 3.1 — Bind one, call one

Solution

```
square = fn x -> x * x end

square.(7)           = 49
is_function(square)   = true
is_function(square, 1) = true
is_function(square, 2) = false

square.(3, 4)
** (BadArityError) #Function<10.129146683 in file:probe_labs7.exs> with arity 1
    called with 2 arguments (3, 4)
```

WHY An anonymous function has exactly one arity, and `is_function/2` tells you what it is without calling it. Wrong count is a `BadArityError`, not a `FunctionClauseError`: nothing was pattern matched, because the call was the wrong shape before any clause could be tried.

Task 3.2 — Forget the dot

```
square = fn x -> x * x end
square(7)

error: undefined function square/1 (there is no such import)
```

WHY `square(7)` means "the named function `square/1`, in this module or an import". The variable called `square` is a different namespace entirely. The dot is how you say "call the value in this variable".

Task 3.3 — A multi-clause fn

FizzBuzz in one anonymous function, mapped over `1..15`.

Solution

```
fizz = fn
  n when rem(n, 15) == 0 -> "FizzBuzz"
  n when rem(n, 3) == 0 -> "Fizz"
  n when rem(n, 5) == 0 -> "Buzz"
  n -> Integer.to_string(n)
end

Enum.map(1..15, fizz)
#=> ["1", "2", "Fizz", "4", "Buzz", "Fizz", "7", "8", "Fizz", "Buzz",
    "11", "Fizz", "13", "14", "FizzBuzz"]
```

WHY Clauses and guards work in `fn` exactly as they do in `def` — top to bottom, first match wins, so the 15 clause must come first. Note `fizz` is passed with no dot and no parentheses: you are handing over the value, not calling it.

Task 3.4 — Functions in a map

Build a calculator: a map from operator strings to two-argument functions, and a `calc` that looks the operator up. Division by zero and an unknown operator must both come back as `{:error, _}`.

Solution

```
ops = %{
  "+" => fn a, b -> a + b end,
  "-" => fn a, b -> a - b end,
  "*" => fn a, b -> a * b end,
  "/" => fn
    _, 0 -> {:error, :div_by_zero}
    a, b -> a / b
  end
}

calc = fn a, op, b ->
  case Map.fetch(ops, op) do
    {:ok, f} -> f.(a, b)
    :error -> {:error, {:unknown_op, op}}
  end
end

calc.(6, "*", 7) = 42
calc.(10, "-", 4) = 6
calc.(1, "/", 4) = 0.25
calc.(1, "/", 0) = {:error, :div_by_zero}
calc.(2, "^", 3) = {:error, {:unknown_op, "^"}}
```

WHY A map of functions is a dispatch table: adding an operator is adding a key, and there is no case statement that grows with it. The division clause pattern matches on `0` — the zero check happens in the head, not in an `if`.

Task 3.5 — A function that returns a function

Solution

```
greeter_for = fn
  :formal -> fn name -> "Good day, #{name}." end
  :casual -> fn name -> "hey #{name}" end
end

greeter_for.(:formal).("Ada") = "Good day, Ada."
greeter_for.(:casual).("Ada") = "hey Ada"
```

WHY Two dots, two calls: the first returns a function, the second calls it. This is the shape every factory in lab 5 and lab 10 has.

Task 3.6 — Clauses of different arity

```
f = fn
  x -> x
  x, y -> x + y
end

error: cannot mix clauses with different arities in anonymous functions
```

WHY A named function can have `f/1` and `f/2` side by side because they are two functions. An anonymous function is ONE value with ONE arity, so there is nowhere to put the second.

CHECKPOINT You can say why `square(7)` fails to compile but `square.(7)` runs, and what arity `greeter_for` has.

LAB 4 · 30 MINUTES · CAPTURE

The capture operator

GOAL Stop writing `fn x -> f(x) end`. The capture operator turns a named function into a value, and turns a short expression into a function.

Task 4.1 — Rewrite each with a capture

The end of `function_lab.exs` has four calls written with `fn`. Rewrite each one.

Solution

```
Enum.map(["a", "b"], fn s -> String.upcase(s) end) = ["A", "B"]
Enum.map(["a", "b"], &String.upcase/1)           = ["A", "B"]
Enum.map([3, 1], [2], &Enum.sort/1)               = [[1, 3], [2]]
Enum.reduce([1, 2, 3], 0, &+/2)                   = 6
Enum.filter([1, :a, 2.0, "b"], &is_number/1)      = [1, 2.0]

inspect(&String.upcase/1)                         = &String.upcase/1
:erlang.fun_info(&String.upcase/1, :type) = {:type, :external}
:erlang.fun_info(fn x -> x end, :type)   = {:type, :local}
```

WHY `&String.upcase/1` is a NAME — module, function, arity — so it prints as itself and its type is `:external`. An `fn` is compiled code with an environment, type `:local`. And `&+/2` is the `+` operator as a function: every operator is a Kernel function, which is session 4's whole point.

Task 4.2 — The shorthand

Solution

```
Enum.map(1..4, &(&1 * &1)) = [1, 4, 9, 16]
Enum.map([a: 1, b: 2], &{elem(&1, 1), elem(&1, 0)}) = [{1, :a}, {2, :b}]
Enum.map([1, 2], &[&1, -&1]) = [[1, -1], [2, -2]]
Enum.zip_with([1, 2], [10, 20], &(&1 + &2)) = [11, 22]
Enum.map(["a-b", "c-d"], &String.split(&1, "-")) = [{"a", "b"}, {"c", "d"}]
Enum.map([1, 2], &{:ok, &1}) = [ok: 1, ok: 2]
```

WHY `&1` is the first argument, `&2` the second, and the arity is the highest one used. Tuples and lists need no extra parentheses. And `[ok: 1, ok: 2]` — a list of two-element tuples whose first element is an atom — IS a keyword list, so that is how it prints.

Task 4.3 — Three captures that do not compile

```
f = &(&2 * 2)
error: capture argument &2 cannot be defined without &1 (you cannot skip
arguments, all arguments must be numbered)

f = &Enum.map(&1, &(&1 + 1))
error: nested captures are not allowed. You cannot define a function using the
capture operator & inside another function defined via &. Got invalid nested
capture: &(&1 + 1)

f = &(:ok)
error: invalid args for &, expected one of:
  * &Mod.fun/arity to capture a remote function, such as &Enum.map/2
```

```
* &fun/arity to capture a local or imported function, such as &is_atom/1
* &some_code(&1, ...) containing at least one argument as &1, such as
  &List.flatten(&1)
Got: :ok
```

WHY The nested one is the one you will actually hit. Inside the inner `&`, `&1` would be ambiguous — which function's first argument? The fix is to make the outer one a plain fn: `fn list -> Enum.map(list, &(&1 + 1)) end`.

Task 4.4 — Capturing a local, and a private, function

Solution

```
defmodule Prices do
  def with_tax(prices), do: Enum.map(prices, &add_tax/1)
  defp add_tax(p), do: Float.round(p * 1.18, 2)
end

Prices.with_tax([100, 250.5]) = [118.0, 295.59]
```

WHY `&add_tax/1` captures a function of THIS module — and it may be private, because the capture is made inside the module. The function value can then travel anywhere, including into `Enum`, and still call the private function.

Task 4.5 — Capture something that does not exist

```
f = &String.upcase/3
IO.puts("created fine: #{inspect(f)}")
f("a", "b", "c")

created fine: &String.upcase/3
** (UndefinedFunctionError) function String.upcase/3 is undefined or private.
Did you mean:

  * upcase/1
  * upcase/2
```

WHY A remote capture is only a name, so creating it checks nothing. The function is looked up when it is CALLED — and the error even suggests the arities that do exist.

CHECKPOINT You can rewrite `fn x -> {:ok, x} end` with a capture, and say why `&Enum.map;(&1, &(&1 + 1))` is a compile error.

LAB 5 · 35 MINUTES · CLOSURES

Closures and higher-order functions

GOAL See exactly what a closure holds — a copy of the values it refers to — and write the functions that take and return functions.

Task 5.1 — Capture, then rebind

Solution

```
rate = 0.18
tax = fn amount -> Float.round(amount * rate, 2) end
tax.(100) = 18.0

rate = 0.05
tax.(100) = 18.0
rate      = 0.05
```

WHY The function captured the VALUE 0.18 when it was created. `rate = 0.05` later binds the name `rate` to a new value; it does not change the old one, and nothing can — data is immutable. So the closure has nothing to notice.

Task 5.2 — A function factory

Solution

```
multiplier = fn n -> fn x -> x * n end end
double = multiplier.(2)
triple = multiplier.(3)

double.(21) = 42
triple.(21) = 63
:erlang.fun_info(double, :env) = {:env, [2]}
:erlang.fun_info(triple, :env) = {:env, [3]}
double == triple? false
```

WHY There it is, literally: the same compiled code, with a different environment. `:env` is a list of the captured values. That is all a closure is.

Task 5.3 — What a closure carries

Solution

```
:erts_debug.size(fn x -> x end)      = 2 words
:erts_debug.size(double)            = 3 words

big = Enum.to_list(1..10_000)
:erts_debug.size(fn -> length(big) end) = 20003 words
:erts_debug.size(fn -> n end, n = length(big)) = 3 words
```

WHY A closure copies what it captures. The first `fn` holds the whole ten-thousand-element list — about 160 KB — even though it only ever wants the length. Compute the small thing first and capture that. This matters most when a closure is sent to another process, because the whole environment goes with it.

Task 5.4 — The counter that does not count

```
count = 0
inc = fn -> count = count + 1 end
inc.()
inc.()
IO.puts("count = #{count}")

warning: variable "count" is unused (there is a variable with the same name in
the context, use the pin operator (^) to match on it or prefix this variable
with underscore if it is not meant to be used)

count = 0
```

WHY The count inside the function is a NEW variable, local to that function, that happens to shadow the outer one. The compiler notices you never use it. State that changes over time lives in a process — GenServer, lab 9 — not in a closure.

Task 5.5 — Higher-order functions you write

Solution

```
defmodule HOF do
  def twice(f), do: fn x -> f.(f.(x)) end
  def compose(f, g), do: fn x -> g.(f.(x)) end
  def apply_n(_f, 0, x), do: x
  def apply_n(f, n, x) when n > 0, do: apply_n(f, n - 1, f.(x))
  def flip(f), do: fn a, b -> f.(b, a) end
end

HOF.twice(&(&1 + 10)).(1)           = 21
HOF.compose(&String.trim/1, &String.length/1).(" hi ") = 2
HOF.apply_n(&(&1 * 2), 10, 1)      = 1024
HOF.flip(&-/2).(1, 10)             = 9
HOF.flip(&Kernel.<=>/2).("world", "hello ") = "hello world"
```

WHY twice and compose RETURN functions; apply_n TAKES one and calls it n times. flip is the odd one out and worth a second look: it takes a two-argument function and returns one that takes the arguments the other way round.

CHECKPOINT You can say what :erts_debug.size would print for a closure over a 1,000-element list, and why the counter stayed at zero.

LAB 6 · 35 MINUTES · ENUM

map, filter, reduce

GOAL Replace loops with Enum's three workhorses — and see that the first two are special cases of the third.

Task 6.1 — Totals from a list of orders

Given five orders — maps with `:id`, `:customer`, `:total` and `:status` — list the totals, the ids of the paid ones, and the paid revenue.

Solution

```
orders = [
  %{id: 1, customer: "ada", total: 120, status: :paid},
  %{id: 2, customer: "grace", total: 80, status: :pending},
  %{id: 3, customer: "ada", total: 45, status: :paid},
  %{id: 4, customer: "linus", total: 300, status: :refunded},
  %{id: 5, customer: "grace", total: 60, status: :paid}
]
paid = Enum.filter(orders, &(&1.status == :paid))

Enum.map(orders, & &1.total)           = [120, 80, 45, 300, 60]
paid |> Enum.map(& &1.id)              = [1, 3, 5]
Enum.reduce(paid, 0, fn o, acc -> acc + o.total end) = 225
... same with Enum.sum_by(paid, & &1.total)      = 225
```

WHY `& &1.total` is the shortest capture there is: a function that returns its argument's `:total` field. The `reduce` function receives the ELEMENT first and the accumulator second — the order people most often get backwards.

Task 6.2 — Reduce into a map

Paid revenue per customer, in one reduce.

Solution

```
orders
|> Enum.filter(&(&1.status == :paid))
|> Enum.reduce(%{}, fn o, acc ->
  Map.update(acc, o.customer, o.total, &(&1 + o.total))
end)

#=> %{"ada" => 165, "grace" => 60}
```

WHY The accumulator can be any shape — here a map. `Map.update/4` inserts the initial value for a new key and applies the function to an existing one. Notice the capture inside the reduce's `fn`: allowed, because the outer function is an `fn`, not a `&`.

Task 6.3 — Map over a map

Solution

```
prices = %{apple: 30, pear: 45}

Enum.map(prices, fn {k, v} -> {k, v * 2} end) = [apple: 60, pear: 90]
Map.new(prices, fn {k, v} -> {k, v * 2} end)  = %{apple: 60, pear: 90}
```

```
for {k, v} <- prices, into: %{}, do: {k, v * 2} = %{apple: 60, pear: 90}
```

WHY Enum.map always returns a LIST. A map goes in as {key, value} tuples and comes out as a list of them, which prints as a keyword list. To get a map back, say so: Map.new/2, or a comprehension with into: %{}.

Task 6.4 — Write map and filter with reduce

Solution

```
defmodule MyEnum do
  def map(list, f) do
    list |> Enum.reduce([], fn x, acc -> [f.(x) | acc] end) |> Enum.reverse()
  end

  def filter(list, keep?) do
    list
    |> Enum.reduce([], fn x, acc -> if keep?.(x), do: [x | acc], else: acc end)
    |> Enum.reverse()
  end
end

MyEnum.map([1, 2, 3], &(&1 * 10))      = [10, 20, 30]
MyEnum.filter(1..10, &(rem(&1, 3) == 0)) = [3, 6, 9]
without the reverse: [30, 20, 10]
```

WHY Prepending with [x | acc] is constant time; appending with acc ++ [x] copies the whole list every time. So you build backwards and reverse once. Lab 8 meets exactly this shape again as tail recursion.

Task 6.5 — reduce with no starting value

Solution

```
Enum.reduce([4, 9, 2], &max/2) = 9
Enum.reduce([], &max/2)
** (Enum.EmptyError) empty error
Enum.reduce([], 0, &max/2) = 0
```

WHY reduce/2 uses the first element as the accumulator — neat, until the list is empty and there is no first element. If the input can ever be empty, give reduce an initial value.

CHECKPOINT You can write Enum.filter/2 using only Enum.reduce/3, and say why you reverse at the end.

LAB 7 · 35 MINUTES · PIPE

The pipe and composition

GOAL Write transformations in the order they happen, and build pipelines from lists of functions when the steps are only known at run time.

Task 7.1 — Unnest it

Turn a title into a URL slug. Write it nested first, then as a pipeline.

Solution

```
title = "  Functions Are Values, Too!  "

Enum.join(String.split(String.downcase(
  String.replace(String.trim(title), ~r/^[^\w\s]/u, ""))), "-")

title
|> String.trim()
|> String.replace(~r/^[^\w\s]/u, "")
|> String.downcase()
|> String.split()
|> Enum.join("-")

nested   = "functions-are-values-too"
piped    = "functions-are-values-too"
equal? true

Macro.expand(quote(do: a |> f(b) |> g()), __ENV__) -> g(a |> f(b))
```

WHY The nested version reads inside out; the pipe reads in the order the work happens. The expansion shows the pipe is a macro: the outer `|>` becomes `g( . . . )` with the rest passed as its first argument, and the inner one unfolds the same way when it is expanded in turn.

Task 7.2 — Pipe into an anonymous function

```
shout = fn s -> String.upcase(s) <> "!" end
"hey" |> shout

error: undefined function shout/1 (there is no such import)
```

Solution

```
"hey" |> shout.()      = "HEY!"
"hey" |> then(shout)    = "HEY!"
"hey" |> then(&(&1 <> &1)) = "heyhey"
```

WHY The pipe rewrites `x |> shout` into `shout(x)` — a call to a NAMED function. Add the dot to call the value, or use `then/2`, which exists for exactly this: it passes the value to any function and returns the result.

Task 7.3 — The wrong argument

Solution

```
"ab" |> String.duplicate(3)      = "ababab"
3 |> String.duplicate("ab")
** (ArgumentError) errors were found at the given arguments:
3 |> then(&String.duplicate("ab", &1)) = "ababab"
```

WHY The pipe always fills the FIRST argument, which is why Elixir's libraries put the data being transformed first. When it is not first, then/2 with a capture puts it wherever you need it.

Task 7.4 — Look without touching

Solution

```
[5, 3, 8]
|> Enum.sort()
|> tap(&IO.puts(" sorted: #{inspect(&1)}"))
|> Enum.sum()

sorted: [3, 5, 8]
result: 16
```

WHY tap/2 calls the function for its side effect and passes the ORIGINAL value on. It is the logging tool for the middle of a pipeline; then/2 is the transforming one.

Task 7.5 — A pipeline you build at run time

Put the steps in a list, then fold the input through them. Then build a single reusable function out of the same list.

Solution

```
steps = [
  &String.trim/1,
  &String.downcase/1,
  &String.replace(&1, ~r/\s+/, " "),
  &String.split(&1, " "),
  &Enum.map(&1, fn w -> String.capitalize(w) end),
  &Enum.join(&1, " ")
]

run_all = fn input, fs -> Enum.reduce(fs, input, fn f, acc -> f.(acc) end) end

compose_all = fn fs ->
  Enum.reduce(fs, &Function.identity/1, fn f, acc -> fn x -> f.(acc.(x)) end end)
end

run_all(" HELLO WORLD ", steps) = "Hello World"
title_case = compose_all.(steps); title_case(" the BEAM ") = "The Beam"
```

WHY Look at the fifth step: fn inside &Enum.map; is allowed because the inner one is a plain fn, not a second &. run_all applies the steps now; compose_all returns one function that applies them later — the same list, composed two ways. Function.identity/1 is the neutral starting point, a function that returns its argument.

Task 7.6 — Composition when a step can fail

Validate signup parameters with three steps. Each returns `{:ok, params}` or `{:error, reason}`; the first error must stop the rest.

Solution

```
defmodule Signup do
  def validate(params) do
    {:ok, params}
    |> step(&required(&1, :email))
    |> step(&has_at/1)
    |> step(&min_age(&1, 18))
  end

  defp step({:ok, p}, f), do: f.(p)
  defp step(error, _f), do: error

  defp required(p, k),
    do: if(Map.has_key?(p, k), do: {:ok, p}, else: {:error, {:missing, k}})
  defp has_at(%{email: e} = p),
    do: if(String.contains?(e, "@"), do: {:ok, p}, else: {:error, :bad_email})
  defp min_age(%{age: a} = p, min) when a >= min, do: {:ok, p}
  defp min_age(_p, min), do: {:error, {:under, min}}
end

Signup.validate(%{email: "a@b.io", age: 30}) = {:ok, %{email: "a@b.io", age: 30}}
Signup.validate(%{age: 30}) = {:error, {:missing, :email}}
Signup.validate(%{email: "nope", age: 30}) = {:error, :bad_email}
Signup.validate(%{email: "a@b.io", age: 12}) = {:error, {:under, 18}}
```

WHY All the plumbing is in the two clauses of `step/2`: an `{:ok, p}` runs the next function, anything else passes straight through. That is the idea with (session 3) turns into syntax — write this version once and with will never look like magic again.

CHECKPOINT You can say what `x |> f(a)` expands to, and when you would reach for `Enum.reduce(steps, x, ...)` instead of a plain pipe.

LAB 8 · 40 MINUTES · RECURSION

Recursion and tail recursion

GOAL Loop the way Elixir loops — a clause for the end, a clause for the rest — and measure the one difference between body and tail recursion instead of taking it on faith.

Task 8.1 — Body recursion, no Enum

In `recursion.exs`, write `sum/1`, `len/1` and `largest/1` over a list.

Solution

```
defmodule ListOps do
  def sum([], do: 0)
  def sum([h | t], do: h + sum(t))

  def len([], do: 0)
  def len([_ | t], do: 1 + len(t))

  def largest([h | t], do: largest(t, h))
  defp largest([], best), do: best
  defp largest([h | t], best) when h > best, do: largest(t, h)
  defp largest([_ | t], best), do: largest(t, best)
end

ListOps.sum([4, 8, 15, 16, 23, 42]) = 108
ListOps.len([:a, :b, :c])           = 3
ListOps.largest([3, 9, 2, 7])       = 9
ListOps.largest([])
** (FunctionClauseError) no function clause matching in ListOps.largest/1
```

WHY There is no clause for the empty list in the public `largest/1`, on purpose: an empty list has no largest element, and the `FunctionClauseError` says so more honestly than a made-up `nil` would.

WATCH OUT Call it `max` and the module does not compile: "imported `Kernel.max/2` conflicts with local function". Every module imports `Kernel`, and your private `max/2` collides with its `max/2` — session 6's `import Kernel, except: [max: 2]` is the other way out.

Task 8.2 — Tail recursion, with an accumulator

Solution

```
defmodule Tail do
  def sum(list), do: sum(list, 0)
  defp sum([], acc), do: acc
  defp sum([h | t], acc), do: sum(t, acc + h)

  def len(list), do: len(list, 0)
  defp len([], n), do: n
  defp len([_ | t], n), do: len(t, n + 1)

  def reverse(list), do: reverse(list, [])
  defp reverse([], acc), do: acc
  defp reverse([h | t], acc), do: reverse(t, [h | acc])
end
```

```
def fact(n) when is_integer(n) and n >= 0, do: fact(n, 1)
defp fact(0, acc), do: acc
defp fact(n, acc), do: fact(n - 1, acc * n)
end

Tail.sum([4, 8, 15, 16, 23, 42]) = 108
Tail.len([:a, :b, :c])          = 3
Tail.reverse([1, 2, 3])         = [3, 2, 1]
Tail.fact(25)                   = 15511210043330985984000000
Tail.__info__(:functions)       = [fact: 1, len: 1, reverse: 1, sum: 1]
```

WHY The pattern: a public function with the natural arity, and a private one with an extra accumulator argument. Only the public ones are exported. In `sum(t, acc + h)` the recursive call is the LAST thing that happens; in `h + sum(t)` the last thing is the `+`, which has to wait for the recursive call to come back.

Task 8.3 — Measure the stack

`StackPeak.measure/1` is given in `recursion.exs`: it runs a function in a fresh process and samples that process's stack. Uncomment the loop at the bottom and run it.

```
n = 250000 body peak = 499821 words tail peak = 3 words
n = 500000 body peak = 998931 words tail peak = 3 words
n = 1000000 body peak = 1999301 words tail peak = 3 words
```

WHY Body recursion needs about two words of stack per element — each pending `h + ...` waits there — so a million elements is roughly sixteen megabytes of stack. The tail version holds three words whatever the length, because each call replaces the one before it. The sampler can miss the very top of the peak, which is why the body column falls a few hundred to a thousand words short of exactly $2n$.

Task 8.4 — Building a list: is tail always faster?

Solution

```
defmodule Doubler do
  def body([], do: [])
  def body([h | t]), do: [h * 2 | body(t)]

  def tail(list), do: tail(list, [])
  defp tail([], acc), do: Enum.reverse(acc)
  defp tail([h | t], acc), do: tail(t, [h * 2 | acc])
end

Doubler.body([1, 2, 3]) = [2, 4, 6]
Doubler.tail([1, 2, 3]) = [2, 4, 6]
forgetting the reverse gives: [6, 4, 2]

1M elements, fresh process each run, best of 9:
Doubler.body 62.6 ms
Doubler.tail 52.1 ms
Enum.map      66.3 ms
ListOps.sum   60.2 ms
Tail.sum      11.7 ms
```

WHY Two different answers. Reducing to one number, the tail version wins by about five times: the body version must grow its stack, and growing means copying. Building a list, the gap closes to about a fifth — the tail version saves the stack but then pays for `Enum.reverse/1`. That is why `Enum.map` is body-recursive, and why "always write tail recursion" is advice to measure, not to follow.

WATCH OUT Your milliseconds will differ, and they vary from run to run on the same machine — deck 7's table shows the spread over eleven runs (the body sum anywhere from 30 to 69 ms). The RATIOS are what hold: tail sum several times faster than body sum; tail map at best about a fifth faster than body map, and on some runs level with it.

Task 8.5 — Recursion over a nested structure

Solution

```
defmodule Tree do
  def flatten([], do: [])
  def flatten([h | t]) when is_list(h), do: flatten(h) ++ flatten(t)
  def flatten([h | t]), do: [h | flatten(t)]

  def depth(list) when is_list(list),
    do: 1 + (list |> Enum.map(&depth/1) |> Enum.max(fn -> 0 end))
  def depth(_), do: 0
end

Tree.flatten([1, [2, [3, [4]], 5], [[6]]]) = [1, 2, 3, 4, 5, 6]
List.flatten(...)                        = [1, 2, 3, 4, 5, 6]
Tree.depth(...)                          = 4
```

WHY A tree is where body recursion is simply the right tool: the recursion follows the shape of the data, and the depth of the stack is the depth of the tree, not its size. The `fn -> 0 end` given to `Enum.max/2` is the answer for an empty list, which has no maximum.

Task 8.6 — A recursive anonymous function

```
fact = fn
  0 -> 1
  n -> n * fact.(n - 1)
end

error: undefined variable "fact"
```

Solution

```
fib = fn
  n, _self when n < 2 -> n
  n, self -> self.(n - 1, self) + self.(n - 2, self)
end

fib.(20, fib) = 6765
```

WHY While the `fn` is being built, the variable it is about to be bound to does not exist yet, so it cannot refer to itself. Pass it in as an argument. In real code, recursion belongs in a named function — this is to show you why.

CHECKPOINT You can say which of `h + sum(t)` and `sum(t, acc + h)` is a tail call, what the stack measurement showed, and why `Enum.map` is not tail-recursive.

LAB 9 · 40 MINUTES · APPLY · BEHAVIOURS

Dynamic calls and behaviours

GOAL Call a function whose module or name is only known at run time — safely — and then give that pattern a contract the compiler can check.

Task 9.1 — apply/3 and mod.fun()

Solution

```
apply(String, :pad_leading, ["7", 3, "0"]) = "007"

m = String
f = :pad_leading
m.pad_leading("7", 3, "0")                = "007"
apply(m, f, ["7", 3, "0"])                 = "007"
apply(fn a, b -> a * b end, [6, 7])        = 42
apply(String, :pad_leading, ["7", 3])      = " 7"    a default filled the 3rd

apply(String, :no_such, [])
** (UndefinedFunctionError) function String.no_such/0 is undefined or private
```

WHY `m.fun()` needs the function name when you write the code; `apply/3` takes it as data. The arguments are always a LIST, and its length is the arity — which is how the two-element call reached `pad_leading/2` through its default.

Task 9.2 — A safe command dispatcher

Commands arrive as strings. Map the ones you allow to `{module, function}` pairs, check the function exists at the right arity, and apply it. Create no atoms from the input.

Solution

```
defmodule Commands do
  @allowed %{
    "uppercase" => {String, :uppercase},
    "reverse"   => {String, :reverse},
    "length"    => {String, :length}
  }

  def run(name, arg) when is_binary(name) do
    with {:ok, {mod, fun}} <- Map.fetch(@allowed, name),
         true <- function_exported?(mod, fun, 1) do
      {:ok, apply(mod, fun, [arg])}
    else
      _ -> {:error, {:unknown_command, name}}
    end
  end
end

Commands.run("uppercase", "beam") = {:ok, "BEAM"}
Commands.run("reverse", "beam") = {:ok, "maeb"}
Commands.run("length", "hello") = {:ok, 5}
Commands.run("halt", "") = {:error, {:unknown_command, "halt"}}
Commands.run("System.halt", "") = {:error, {:unknown_command, "System.halt"}}
```

```
atoms before 1000 unknown commands: 21159
atoms after:                          21159
```

WHY The lookup is in a map YOU wrote, so the input can only ever choose among functions you listed — "System.halt" is just an unknown string. And the atom count does not move: this is session 5's rule, applied to function names. String.to_atom on user input would be both a security hole and an atom leak.

Task 9.3 — A behaviour and two implementations

Solution

```
defmodule Storage do
  @callback put(key :: String.t(), value :: term()) :: :ok
  @callback get(key :: String.t()) :: {:ok, term()} | :error
  @callback name() :: String.t()
end

defmodule Storage.Memory do
  @behaviour Storage
  @impl true
  def put(_key, _value), do: :ok
  @impl true
  def get("greeting"), do: {:ok, "hello"}
  def get(_), do: :error
  @impl true
  def name, do: "memory"
end

# Storage.Null: the same three callbacks, get/1 always :error

defmodule Cache do
  def lookup(store, key) do
    case store.get(key) do
      {:ok, v} -> "#{store.name()} hit: #{inspect(v)}"
      :error -> "#{store.name()} miss"
    end
  end
end

Cache.lookup(Storage.Memory, "greeting") = "memory hit: \"hello\""
Cache.lookup(Storage.Memory, "other") = "memory miss"
Cache.lookup(Storage.Null, "greeting") = "null miss"
Cache.lookup(Storage.Null, "other") = "null miss"
Storage.behaviour_info(:callbacks) = [get: 1, name: 0, put: 2]
```

WHY Cache never names an implementation: it calls store.get/1 on whatever module it was given. The behaviour is the promise that any such module HAS those functions. A module name is an atom (session 6), so passing one around is as cheap as passing :ok.

Task 9.4 — Break the contract, twice

Write Storage.Broken with no name/0, and Storage.Sloppy with @impl missing on one callback.

```
warning: module attribute @impl was not set for function get/1 callback
```

```
(specified in Storage). This either means you forgot to add the "@impl true"
annotation before the definition or that you are accidentally overriding this
callback
```

```
-> Storage.Sloppy
```

```
warning: function name/0 required by behaviour Storage is not implemented
(in module Storage.Broken)
```

WHY Both are warnings, not errors — the modules compile. The second is the one that saves you: a missing callback would otherwise surface as an `UndefinedFunctionError` the first time someone called it, possibly in production. The first exists because once a module uses `@impl` anywhere, the compiler wants it everywhere, so that a stray function that happens to match a callback is visible.

Task 9.5 — GenServer is a behaviour

Solution

```
defmodule Tally do
  use GenServer

  def start_link(start), do: GenServer.start_link(__MODULE__, start)
  def add(pid, n), do: GenServer.cast(pid, {:add, n})
  def total(pid), do: GenServer.call(pid, :total)

  @impl true
  def init(start), do: {:ok, start}

  @impl true
  def handle_cast({:add, n}, total), do: {:noreply, total + n}

  @impl true
  def handle_call(:total, _from, total), do: {:reply, total, total}
end

{:ok, pid} = Tally.start_link(100)
Tally.add(pid, 5)
Tally.add(pid, 37)
Tally.total(pid) = 142
Tally.__info__(:attributes)[:behaviour] = [GenServer]
```

WHY Here is the counter from lab 5.4 done properly: the state lives in a process, and each callback returns the NEW state. `GenServer` owns the loop and the messages; your module supplies three callbacks. `use GenServer` did the rest — including declaring the behaviour, as the attribute shows.

CHECKPOINT You can say why `Commands.run/2` creates no atoms, and which of the two behaviour warnings would have caught a bug before it shipped.

LAB 10 · 50 MINUTES · CAPSTONE

Capstone: a pricing engine

GOAL Build one small program that uses every idea in the session: a protocol for the items, closures for the rules, a pipeline to apply them, a whitelist to load them from configuration, and a behaviour for the output.

`pricing.exs` gives you the protocol `Priceable`, three item structs — `Book`, `Coffee` and `Voucher` — the `Receipt` behaviour, and a sample cart. It runs as handed out and prints `:todo`. The whole solution is `verify/pricing_solution.exs`; the outputs below are what it prints.

WATCH OUT Structs defined in a script cannot be built at the top level of that same script — "the struct is being accessed in the same context that defines it". That is why the sample cart is built inside `Sample.cart/0`: by the time a function body runs, the struct's module has been compiled.

Task 10.1 — Implement the protocol

Solution

```
defimpl Priceable, for: Book do
  def price(%Book{cents: c}), do: c
  def label(%Book{title: t}), do: "Book: #{t}"
end

defimpl Priceable, for: Coffee do
  def price(%Coffee{size: :small, qty: q}), do: 250 * q
  def price(%Coffee{size: :large, qty: q}), do: 400 * q
  def label(%Coffee{size: s, qty: q}), do: "Coffee #{s} x#{q}"
end

defimpl Priceable, for: Voucher do
  def price(%Voucher{cents: c}), do: -c
  def label(%Voucher{code: code}), do: "Voucher #{code}"
end

Book: Programming Elixir      4200
Coffee large x2                800
Coffee small x3               750
Book: Designing for Scale     3150
Voucher WELCOME               -500

Priceable.price(:oops)
** (Protocol.UndefinedError) protocol Priceable not implemented for Atom
```

WHY The protocol dispatches on the item's TYPE; within the Coffee implementation, ordinary clauses dispatch on its size. Money is integer cents throughout — no floats anywhere near a price.

Task 10.2 — Rules are closures

Each factory returns a RULE: a one-argument function from cents to cents.

Solution

```
defmodule Rules do
  def percent_off(pct) when pct in 0..100,
    do: fn cents -> cents - div(cents * pct, 100) end
end
```

```
def flat_off(off) when off >= 0, do: fn cents -> cents - off end
def at_least(floor), do: fn cents -> max(cents, floor) end

def over(threshold, rule),
  do: fn cents -> if cents >= threshold, do: rule.(cents), else: cents end
end

ten = Rules.percent_off(10)
Rules.percent_off(10).(10_000)      = 9000
Rules.flat_off(300).(10_000)        = 9700
Rules.at_least(1000).(400)          = 1000
Rules.over(5000, ten).(4_000)       = 4000
Rules.over(5000, ten).(6_000)       = 5400
:erlang.fun_info(ten, :env)         = {:env, [10]}
```

WHY `over/2` is the higher-order one: it takes a rule and returns a rule. `{:env, [10]}` is lab 5.2 again — the percentage IS the closure.

WATCH OUT Print that `:env` with a plain `inspect` and you get `{:env, ~c"\n"}` — a list of small integers that are all printable is shown as a charlist, and 10 is a newline. The value is `[10]`; pass char lists: `:as_lists` to see it that way.

Task 10.3 — Checkout

Solution

```
defmodule Checkout do
  def subtotal(cart), do: subtotal(cart, 0)
  defp subtotal([], acc), do: acc
  defp subtotal([item | rest], acc), do: subtotal(rest, acc + Priceable.price(item))

  def total(cart, rules) do
    cart
    |> subtotal()
    |> apply_rules(rules)
  end

  def apply_rules(cents, rules),
    do: Enum.reduce(rules, cents, fn rule, acc -> rule.(acc) end)
end

rules = [Rules.percent_off(10), Rules.flat_off(300), Rules.at_least(1000)]

Checkout.subtotal(cart)              = 8400
Checkout.total(cart, rules)          = 7260
Checkout.total(cart, Enum.reverse(rules)) = 7290  order matters
Checkout.total([], rules)            = 1000
Checkout.total(cart, [])             = 8400
```

WHY Tail recursion for the subtotal, a pipe for the order of work, a reduce over a list of functions for the rules. 10% then 300 off gives 7260; 300 off then 10% gives 7290 — so rule order is part of the business logic and belongs in configuration. And an empty cart costs 1000, because the floor rule says so: decide whether that is what the business wants.

Task 10.4 — Rules from configuration

Configuration arrives as strings: ["percent:10", "flat:300", "min:1000"]. Turn it into rules — or a clear error — with no crash for bad input and no atoms created.

Solution

```
# inside Rules
@factories %{
  "percent" => {&__MODULE__.percent_off/1, &__MODULE__.percentage?/1},
  "flat" => {&__MODULE__.flat_off/1, &__MODULE__.non_negative?/1},
  "min" => {&__MODULE__.at_least/1, &__MODULE__.non_negative?/1}
}

def percentage?(n), do: n in 0..100
def non_negative?(n), do: n >= 0

def from_config(entries), do: from_config(entries, [])
defp from_config([], acc), do: {:ok, Enum.reverse(acc)}
defp from_config([entry | rest], acc) do
  with [name, arg] <- String.split(entry, ":", parts: 2),
       {:ok, {factory, valid?}} <- Map.fetch(@factories, name),
       {n, ""} <- Integer.parse(arg),
       true <- valid?.(n) do
    from_config(rest, [factory.(n) | acc])
  else
    _ -> {:error, {:bad_rule, entry}}
  end
end

Rules.from_config(Sample.config()) -> :ok, 3 rules
total with configured rules          = 7260
same as the hand-built list?         true
Rules.from_config(["percent:10", "halt:1"]) = {:error, {:bad_rule, "halt:1"}}
Rules.from_config(["percent:ten"]) = {:error, {:bad_rule, "percent:ten"}}
Rules.from_config(["flat"]) = {:error, {:bad_rule, "flat"}}
Rules.from_config(["percent:200"]) = {:error, {:bad_rule, "percent:200"}}
Rules.from_config(["flat:-5"]) = {:error, {:bad_rule, "flat:-5"}}
atoms created by 500 unknown rule names: 0
```

WHY The map holds PAIRS of functions — a factory and its validator — so a new rule type is one line. Without the validator, "percent:200" reaches percent_off/1's guard and crashes with a `FunctionClauseError`: the guard is a last line of defence, not input validation. The accumulator is reversed at the end, so the rules keep the order they were written in.

WATCH OUT The obvious first version puts the validators in the map as anonymous functions — `&(&1 in 0..100)` — and fails to compile: "cannot inject attribute `@factories` into function/macro because cannot escape `#Function<...>`". The supported values are: lists, tuples, maps, atoms, numbers, bitstrings, PIDs and remote functions in the format `&Mod.fun/arity`. An attribute is written into the compiled code, and only a remote capture — a name — can be written there. That is why the validators are named functions.

Task 10.5 — Receipts through a behaviour

Write `Receipt.Text` and `Receipt.Csv`, both adopting `@behaviour Receipt`. Choose between them at run time from a string.

Solution

```
# inside Checkout
@formats %{"text" => Receipt.Text, "csv" => Receipt.Csv}

def receipt(cart, rules, format) do
  with {:ok, mod} <- Map.fetch(@formats, format) do
    lines = Enum.map(cart, &{Priceable.label(&1), Priceable.price(&1)})
    {:ok, mod.render(lines, total(cart, rules))}
  else
    :error -> {:error, {:unknown_format, format}}
  end
end

defmodule Receipt.Csv do
  @behaviour Receipt
  @impl true
  def name, do: "csv"
  @impl true
  def render(lines, total) do
    rows = Enum.map(lines, fn {label, c} -> "#{label},#{c}" end)
    Enum.join(["item,cents" | rows] ++ ["TOTAL,#{total}"], "\n")
  end
end

# Receipt.Text: the same shape, padded columns, cents shown as 42.00
```

```
Book: Programming Elixir           42.00
Coffee large x2                     8.00
Coffee small x3                     7.50
Book: Designing for Scale          31.50
Voucher WELCOME                    -5.00
-----
TOTAL                               72.60

item,cents
Book: Programming Elixir,4200
Coffee large x2,800
Coffee small x3,750
Book: Designing for Scale,3150
Voucher WELCOME,-500
TOTAL,7260

Checkout.receipt(cart, rules, "pdf") = {:error, {:unknown_format, "pdf"}}
Receipt.behaviour_info(:callbacks)  = [name: 0, render: 2]
```

WHY A module attribute CAN hold module names — they are atoms — so `@formats` needs no workaround. The protocol answered "what is this item worth" per TYPE of data; the behaviour answers "how should the receipt look" per CHOICE of module. Deck 7's rule of thumb, in one program.

Task 10.6 — Extend it (if there is time)

Pick one. Add a Tip item whose price/1 comes from a percentage of the subtotal — and notice that a protocol cannot see the rest of the cart. Or add a rule type over `:5000:percent:5` that reuses `Rules.over/2`. Or add a "json" receipt without touching `Checkout`, and say what that tells you about where the list of formats should live.

WATCH OUT The third one has an honest answer: you cannot, because `@formats` is a closed list inside `Checkout`. Making it open — a registry, or configuration — is exactly session 6's capstone. Closed lists are safer; open ones are extensible. Choose deliberately.

CHECKPOINT Your engine answers every bad configuration entry with a tagged tuple, creates no atoms from input, and prints the same total — 7260 — whether the rules were built by hand or loaded from strings.

Where this goes next

You can now write, pass, compose and abstract over functions. Session 8 is macros — functions that run at compile time and return code — which is where `defmacro`, `quote` and the use `GenServer` you used in lab 9 stop being something a library hands you and start being something you write.

If you remember one thing per lab	It is this
1 · Named functions	the name is name/arity; defaults generate the short ones
2 · Clauses and guards	first match wins; a guard that raises just fails
3 · Anonymous functions	one arity, called with a dot
4 · Capture	<code>&Mod.fun/arity</code> is a name; <code>&(&1)</code> is sugar for <code>fn</code>
5 · Closures	they copy values, and cannot change anything outside
6 · Enum	<code>map</code> and <code>filter</code> are <code>reduce</code> ; <code>Enum</code> always returns a list
7 · Pipe	<code>x > f(a)</code> is <code>f(x, a)</code> ; a list of functions is a pipeline too
8 · Recursion	tail calls use constant stack; tail is not always faster
9 · Dynamic calls	look the function up in a map you wrote
10 · Capstone	protocols dispatch on data, behaviours on a chosen module

The one-line rule

A function is a value. Keep functions small, name them by what they do, and combine them — with the pipe when the steps are fixed, with `reduce` when they are data, and with a behaviour or a protocol when the implementation must vary.

MEASURED Every output in this guide was produced by `../verify/probe_labs7.exs` and `../verify/pricing_solution.exs` on Elixir 1.20.4 / Erlang/OTP 29. Every compiler error and warning came from a real compile in a child VM. If yours disagree, your machine is right — say so and work out why.